



# Programación

## Interfaces gráficas: Swing I

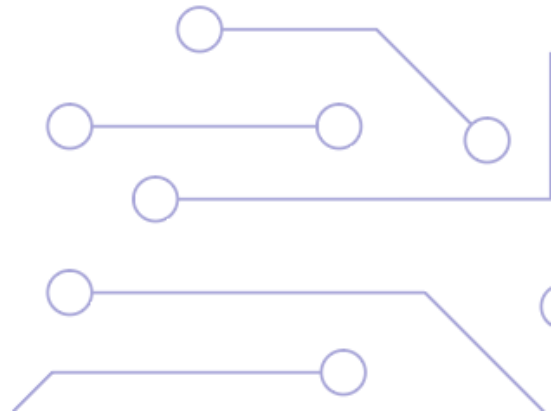
(Contenedores y elementos)

# Interfaces gráficas

- *Graphical User Interface (GUI)*: Es una interfaz con una representación visual de alto nivel que permite al usuario interactuar con la aplicación.
  - Consta de ventanas, botones, menús, barras, etc.
- Por definición, siguen los principios de la Programación Orientada a Objetos debido a su estructura y comportamiento.
- La ejecución no es lineal, se basa en eventos activados por el usuario.
  - Los eventos son asíncronos, es decir, se activan por las acciones del usuario:
    - Al pulsar un botón, hacer clic en un elemento, pulsar una tecla, cambiar el tamaño de la ventana...

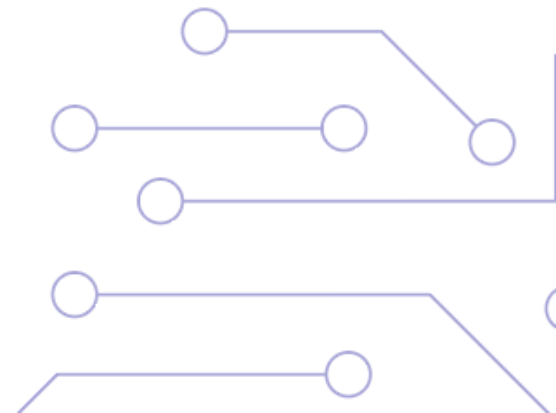
# Swing

- Es una biblioteca de herramientas para crear interfaces gráficas en Java. Su apariencia es independiente de la plataforma en la que se utilice la aplicación.
  - Al igual que otras bibliotecas, su apariencia varía según el sistema operativo.
- Además de elementos simples (botones, etiquetas, casillas de verificación, etc.), ofrece otros elementos de nivel superior: pestañas, tablas, listas....



# Contenedores y elementos

- En Swing, podemos distinguir dos tipos de objetos.
  - Los contenedores: Son objetos que se utilizan para organizar elementos simples dentro de sí mismos siguiendo una maquetación (*layout*). Entre otros, las ventanas son contenedores.
  - Los elementos: Son objetos simples de interfaz gráfica que se organizan en contenedores. Como botones, etiquetas, menús, etc....



# Contenedores: JFrame

- Es una ventana de nivel superior con borde, título e iconos para maximizar, minimizar y cerrar.
- Las aplicaciones gráficas tienen al menos un contenedor de este tipo.
- Un JFrame tiene por defecto un contenedor (*content pane*) para colocar otros elementos.
- Algunos métodos:
  - **JFrame()** y **JFrame(String name)**: Crean una ventana.
  - **getContentPane()**: Obtiene el contenedor de la ventana.
  - **setDefaultCloseOperation(int option)**: El comportamiento que debe tener la ventana al cerrarse (DO\_NOTHING\_ON\_CLOSE | HIDE\_ON\_CLOSE | DISPOSE\_ON\_CLOSE | EXIT\_ON\_CLOSE).

# Contenedores: JFrame

*//Creación*

```
JFrame frame = new JFrame("FrameDemo");
```

*//Comportamiento al cerrar*

```
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

*//Asignación de tamaño*

```
frame.setPreferredSize(new Dimension(300, 200));
```

```
JLabel etiqueta = new JLabel("Hola");
```

```
frame.getContentPane().add(etiqueta, BorderLayout.CENTER);
```

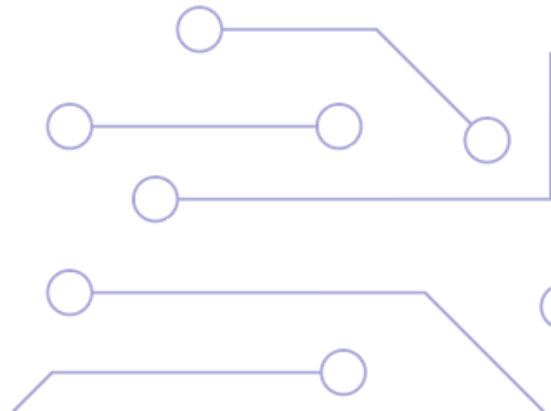
*//Visualización*

```
frame.pack();
```

```
frame.setVisible(true);
```

# Contenedores: JFrame

- Al crear una ventana compleja, conviene crear una nueva clase que extienda la clase JFrame.
  - Los métodos añadir (*add*), quitar (*remove*) y establecer diseño (*setLayout*) están reescritos en el constructor de la clase que la extiende.
  - No es necesario obtener el contenedor (*getContentPane*) para usar estos métodos.



# Contenedores: JFrame

```
public class MiVentana extends JFrame
{
    Miventana()
    {
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setPreferredSize(new Dimension(300,200));
        setLayout(new BorderLayout());
        JButton boton = new JButton("Mi botón");
        add(boton);
        pack();
        setVisible(true);
    }
}
```



# Contenedores: JPanel

- Es un contenedor normalmente invisible donde se colocan los elementos.
- Por defecto, hay uno dentro de un JFrame.
- Métodos:
  - JPanel(): Es un constructor predeterminado.
  - JPanel(maquetacion LayoutManager): Es otro constructor donde se especifica la maquetación (*layout*).
  - void setLayout(maquetacion LayoutManager): Establece la maquetación del contenedor.

# Contenedores: JPanel

- Métodos:
  - void add (Component element): Agrega el elemento pasado como parámetro.
  - void add(Component element, int position): Agrega el elemento pasado como parámetro en la posición especificada.
  - void remove (Component element): Elimina el elemento pasado como parámetro.
  - void remove(int position): Elimina el elemento en la posición pasada como parámetro.
  - Otros métodos: getIcon, setIcon, getWidth, getHeight, setPreferredSize, setBackground.

# Contenedores: JScrollPane

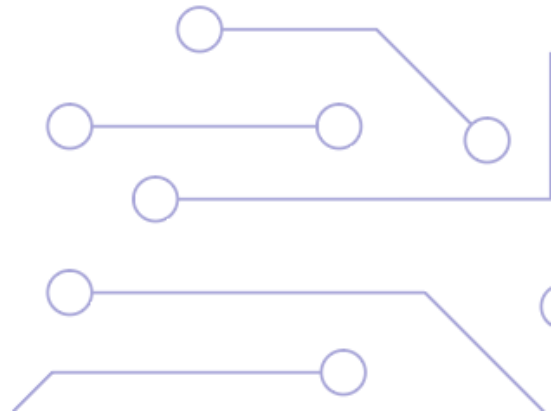
- Es igual que el contenedor anterior, pero permite barras de desplazamiento horizontales y verticales *scroll*.
- Métodos:
  - JScrollPane(ContainerComponent, int scroll\_vertical, int scroll\_horizontal): Constructor, números enteros que establecen la política de scroll, sus valores pueden ser:
    - ScrollPaneConstants.VERTICAL\_SCROLLBAR\_AS\_NEEDED // O HORIZONTAL
    - ScrollPaneConstants.VERTICAL\_SCROLLBAR\_ALWAYS
    - ScrollPaneConstants.VERTICAL\_SCROLLBAR\_NEVER
- Otros métodos: setPreferredSize, setVerticalScrollBarPolicy, setHorizontalScrollBarPolicy, setVisibleRowCount.

# Contenedores: JScrollPane

```
JFrame ventana = new JFrame();
JScrollPane panel1 = new JScrollPane();
JTextArea jt1 = new JTextArea("...", 10,40);
ventana.setTitle("Mi aplicación");
ventana.setDefaultCloseOperation(ventana.EXIT_ON_CLOSE);
panel1.setViewportViewView(jt1);
panel1.setVerticalScrollBarPolicy(ScrollPaneConstants.VERTICAL_SCROLLBAR_ALWAYS);
ventana.getContentPane().add(panel1);
ventana.pack();
ventana.setVisible(true);
```

# Contenedores: JInternalFrame

- Un contenedor que se puede añadir dentro de una ventana padre.
- Se utiliza cuando se desea mostrar más de un documento a la vez.
- Se puede crear mediante los siguientes métodos:
  - JInternalFrame()
  - JInternalFrame(String nombre)
  - JInternalFrame(String nombre, boolean puedeCambiarDetamanyo, boolean puedeCerrarse, boolean puedeExpandirse)

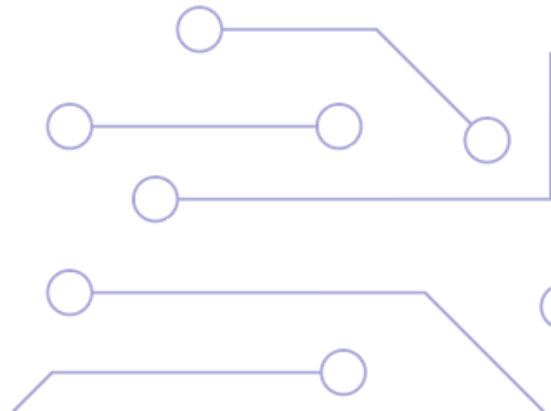


# Contenedores: JInternalFrame

```
MiVentana()
{
    super("Creador de Internal frame");
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setSize(450, 400);
    Container content = getContentPane();
    JDesktopPane desktop = new JDesktopPane();
    content.add(desktop, BorderLayout.CENTER);
    for(int i = 0; i < 5; i++) {
        JInternalFrame frame = new JInternalFrame((" Subventana " + i), true,
            true, true, true);
        frame.setLocation(i * 50 + 10, i * 50 + 10);
        frame.setSize(200, 150);
        frame.setBackground(Color.white);
        frame.setVisible(true);
        desktop.add(frame);
        frame.moveToFront();
    }
    setVisible(true);
}
```

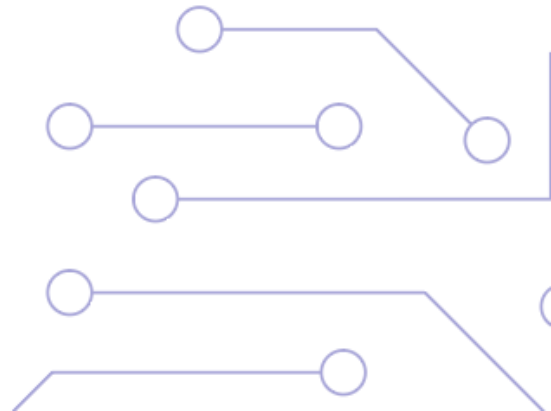
# Contenedores: JDialog

- Es una ventana que permite al usuario introducir algún dato.
- Los tipos de elementos que se pueden usar son limitados; por ejemplo, no pueden incluir menús ni barras de herramientas.
- Se crean adjuntos a un JFrame. Al cerrar el JFrame padre, se cierran todos sus JDialogs.
- Se pueden crear mediante los siguientes métodos:
  - JDialog()
  - JDialog(Frame)
  - JDialog(Frame, String nombre)
  - JDialog(Frame, String nombre, boolean esModal)



# Contenedores: JOptionPane

- Una colección de modales de Jdialog prediseñados.
  - Una ventana es modal si está bloqueada en la aplicación hasta que se cierra.
- Se utilizan para dar avisos al usuario.
- El enumerador de la clase JOptionPane se utiliza para indicar el tipo de ventana.
  - JOptionPane.ERROR\_MESSAGE
  - JOptionPane.INFORMATION\_MESSAGE
  - JOptionPane.WARNING\_MESSAGE
  - JOptionPane.QUESTION\_MESSAGE
  - JOptionPane.PLAIN\_MESSAGE





# Contenedores: JOptionPane

```
JOptionPane.showMessageDialog(null, "Cuidado!", "Error", JOptionPane.ERROR_MESSAGE);
```

```
int opcionelegida = JOptionPane.showConfirmDialog(null, "Adelanta uno", "Opcion", JOptionPane.YES_NO_OPTION);
```

```
String ValordeEntrada = JOptionPane.showInputDialog("Introduce un valor:");
```

# Contenedores: JOptionPane

```
Object[] opt = { "OK", "CANCEL" };  
JOptionPane.showOptionDialog(null, "OK pulsa para continuar", "Aviso", JOptionPane.DEFAULT_OPTION,  
JOptionPane.WARNING_MESSAGE, null, opt, opt [0]);
```

```
Object[] values = {"Uno", "Dos", "Tres"};  
Object valorElegido = JOptionPane.showInputDialog(null, "Elige uno",  
"Entrada",JOptionPane.INFORMATION_MESSAGE, null, values, values[0]);
```

# Contenedores: JFileChooser

- Es un contenedor que permite seleccionar carpetas y archivos del sistema de archivos.
- Métodos:
  - JFileChooser(String nombre\_de\_archivo): Constructor
  - int showOpenDialog(Component): Muestra la ventana de carga.
  - int showSaveDialog(Component): Muestra la ventana de guardado.
  - int showDialog(Component, String nombreVentana): Muestra una ventana personalizada.
    - Estos tres métodos reciben un código de salida, con las opciones FileChooser.APPROVE\_OPTION, JFileChooser.CANCEL\_OPTION y JFileChooser.ERROR\_OPTION.



# Contenedores: JFileChooser

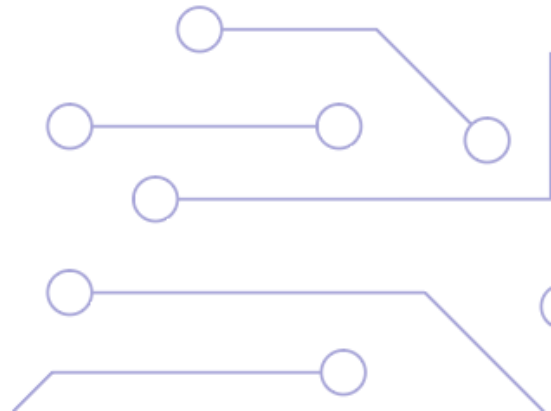
- Métodos:
  - File getSelectedFile(): Devuelve el archivo seleccionado
  - File[] getSelectedFiles(): Devuelve los archivos seleccionados
  - void setSelectedFile(File): Cambia el archivo seleccionado por el que se recibe como parámetro
  - File getCurrentDirectory(): Devuelve la carpeta actual
  - void setMultiSelectionEnabled(boolean): Habilita la selección de múltiples archivos

# Contenedores: JFileChooser

```
JFileChooser fc = new JFileChooser("C:\\");
fc.setMultiSelectionEnabled(true);
int result = fc.showOpenDialog(new JFrame());
if (result == JFileChooser.APPROVE_OPTION)
{
    File file = fc.getSelectedFile();
    ...
}
```

# Contenedores: JSplitPane

- Se utiliza para mostrar dos contenedores en la misma ventana, tanto horizontal como verticalmente.
- Métodos:
  - `JSplitPane(int horizontalOVertical, Component container1, Component container2)`: Crea un contenedor `JsplittedPane` con los dos contenedores pasados como parámetros. El parámetro numérico especifica si se divide horizontal o verticalmente: `JSplitPane.HORIZONTAL_SPLIT` o `JSplitPane.VERTICAL_SPLIT`.



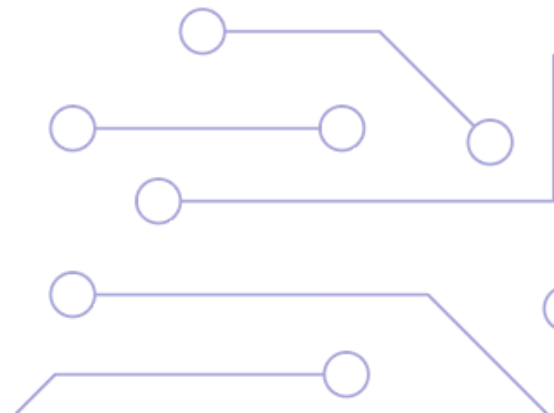
# Contenedores: JTabbedPane

- Es un contenedor que permite organizar varios contenedores en pestañas.
- Métodos:
  - JTabbedPane(): Constructor predeterminado.
  - JTabbedPane(int posicion, int maquetacion): Establece las pestañas en la posición indicada por el primer parámetro y utiliza la maquetación indicada por el segundo parámetro.
    - JTabbedPane.TOP, JTabbedPane.BOTTOM, JTabbedPane.LEFT y JTabbedPane.RIGHT.
    - JTabbedPane.SCROLL\_TAB\_LAYOUT o JTabbedPane.WRAP\_TAB\_LAYOUT.
  - Otros métodos: void addTab(String nombre, Component container), void setSelectedIndex(int pestanyaSeleccionada)...



# Contenedores: JToolBar

- Es un contenedor que muestra varios elementos (normalmente botones con iconos) horizontal o verticalmente en una barra.
- La barra se puede mover a diferentes posiciones dentro del contenedor en el que se encuentra.





# Otros métodos útiles

| Método                                                      | Significado                                                      |
|-------------------------------------------------------------|------------------------------------------------------------------|
| void setVisible(boolean)<br>boolean isVisible()             | Muestra el contenedor.<br>Devuelve si es visible o no.           |
| void setBounds(int, int, int, int)<br>Rectangle getBounds() | Asigna los límites especificados.<br>Devuelve los límites.       |
| void setSize(int, int)<br>Dimension getSize()               | Asigna el tamaño especificado.<br>Devuelve el tamaño.            |
| void setBorder(Border)<br>Border getBorder()                | Asigna la arista especificada.<br>Devuelve la arista que tiene.  |
| void setForeground(Color)<br>Color getForeground()          | Asigna el color (letra) anterior.<br>Devuelve el color anterior. |
| void setBackground(Color)<br>Color getBackground()          | Establece el color de fondo.<br>Devuelve el color de fondo.      |

# Otros métodos útiles

| Método                                                 | Significado                                                                                                                               |
|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| void setFont(Font)<br>Font getFont()                   | Establece el tipo de letra.<br>Devuelve el tipo de letra.                                                                                 |
| void setCursor(Cursor)<br>Cursor getCursor()           | Establece el tipo de cursor.<br>Devuelve el cursor.                                                                                       |
| void setToolTipText(String)<br>String getToolTipText() | Establece el texto que aparece al pasar el ratón sobre un elemento.<br>Devuelve el texto que aparece al pasar el ratón sobre un elemento. |
| void pack()                                            | Adapta el tamaño del contenedor al tamaño de los elementos que contiene.                                                                  |
| Point getLocation()<br>Point getLocationOnScreen()     | Devuelve la posición del contenedor.<br>Devuelve la posición del contenedor en la pantalla.                                               |

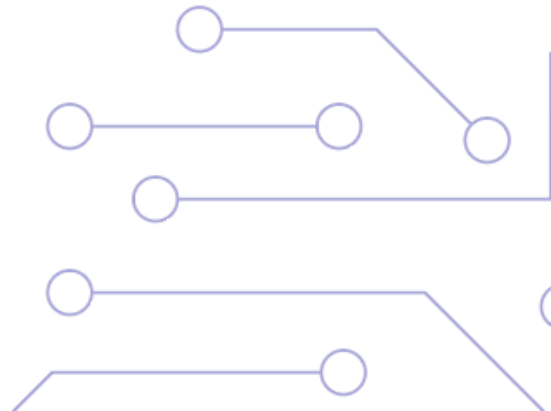
# Otras clases útiles

| Clase     | Constructor / Métodos                                                                                                                                                                                                                                            |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dimension | <code>new Dimension(anchura, altura)</code>                                                                                                                                                                                                                      |
| Point     | <code>new Point(x, y)</code>                                                                                                                                                                                                                                     |
| Rectangle | <code>new Rectangle(x, y, anchura, altura)</code>                                                                                                                                                                                                                |
| Color     | <code>new Color(int rojo, int verde, int azul)</code> : numeros entre 0-255<br><code>new Color(int constante)</code> : le pasa un color constante, <code>Color.WHITE...</code>                                                                                   |
| Font      | <code>new Font("Helvetica", Font.BOLD + Font.ITALIC, 18)</code><br>Constantes: <code>Font.BOLD</code> , <code>Font.ITALIC</code> eta <code>Font.PLAIN</code><br>Algunos otros metodos: <code>getName()</code> , <code>getStyle()</code> , <code>getSize()</code> |
| Cursor    | <code>new Cursor(Cursor.HAND_CURSOR)</code>                                                                                                                                                                                                                      |
| Border    | <code>new BorderFactory.createLineBorder(Color.black))</code>                                                                                                                                                                                                    |
| ImageIcon | <code>new ImageIcon(nombre_de_archivo)</code>                                                                                                                                                                                                                    |

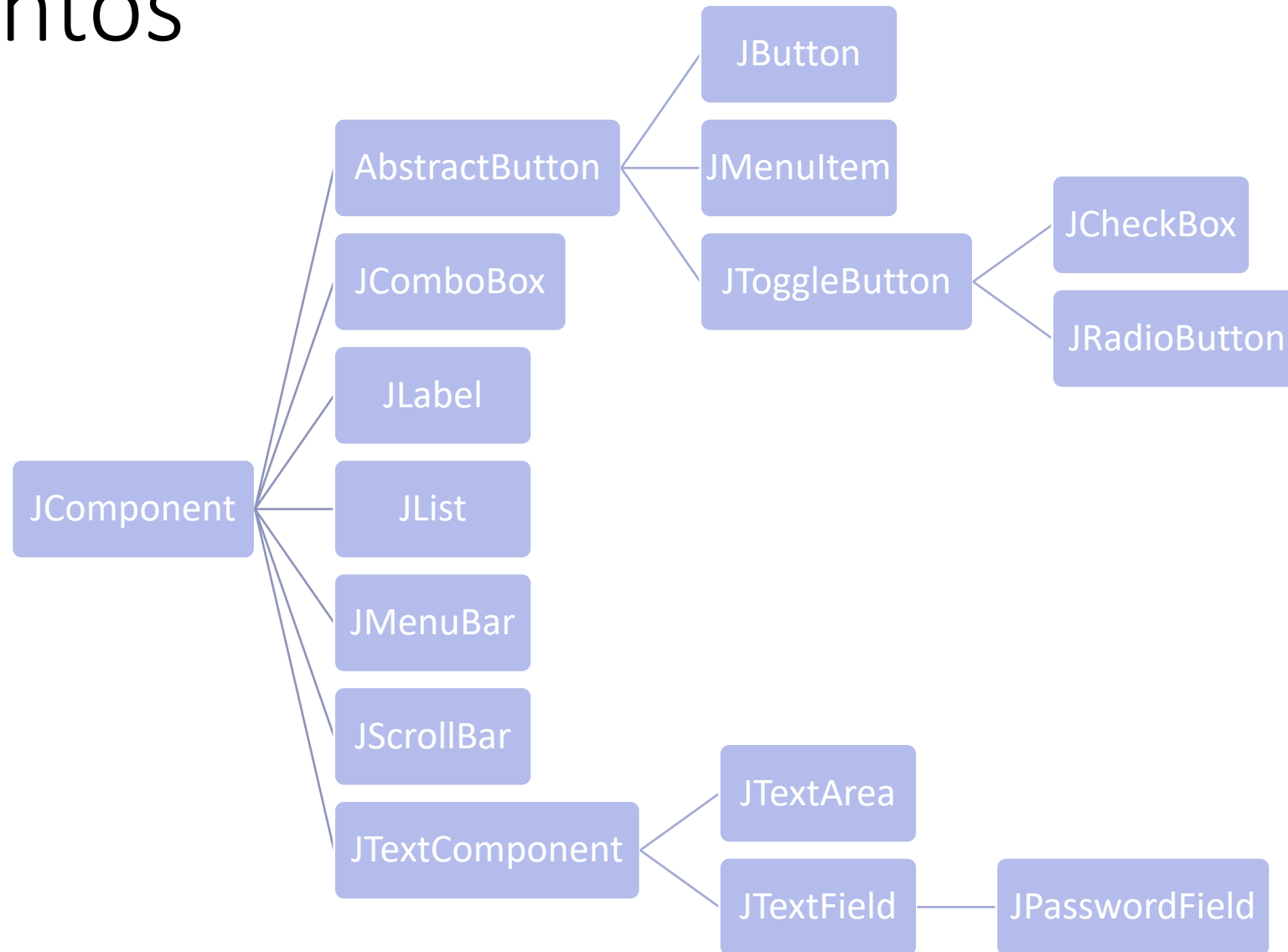


# Elementos

- Objetos simples de interfaz gráfica
- Deben colocarse dentro de un contenedor para ser mostrados.
- Todos derivan del paquete `javax.swing.JComponent`.
  - También los contenedores.

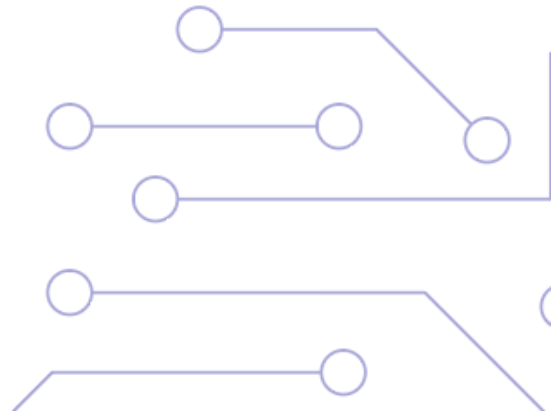
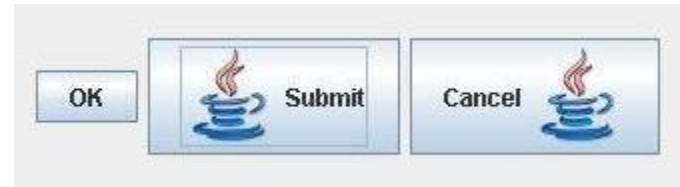


# Elementos



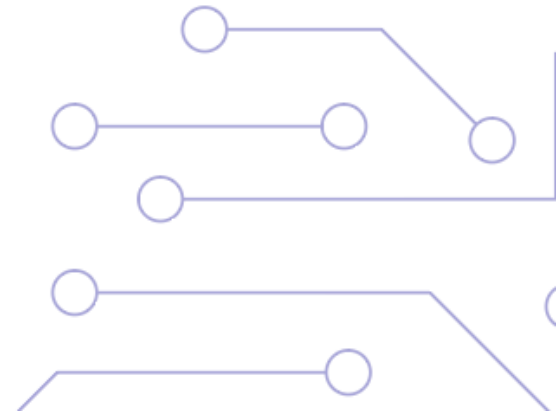
# Elementos: JButton

- Un botón que puede contener texto, una imagen o ambos.
- Métodos:
  - JButton()
  - JButton(Icon)
  - JButton(String)
  - JButton(String, Icon)
  - void setText(String)
  - String getText()
  - setToolTipText(String)
  - String getToolTipText()
  - setIcon(Icon)
  - Icon getIcon()



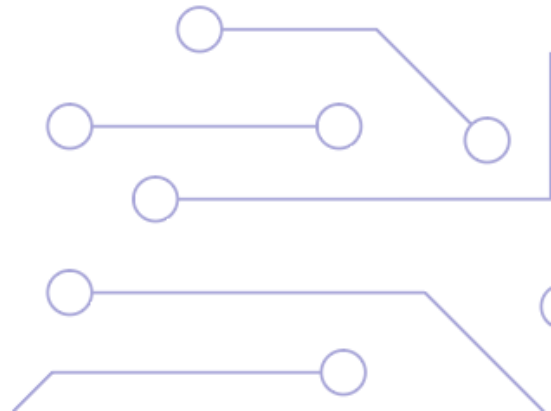
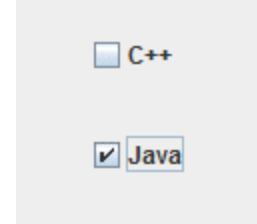
# Elementos: JToggleButton

- Un botón con dos estados, pulsado y no pulsado.
- Tiene los mismos métodos que un botón normal y algunos métodos propios
- Métodos:
  - JToggleButton()
  - JToggleButton(Icon)
  - JToggleButton(Icon, boolean)
  - JToggleButton(String)
  - JToggleButton(String, boolean)
  - JToggleButton(String, Icon, boolean)
  - boolean isSelected()
  - void setSelected(boolean)



# Elementos: JCheckBox

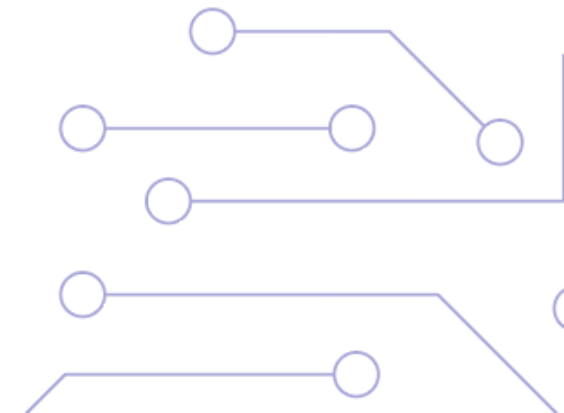
- Se utiliza para expresar múltiples opciones.
- Métodos:
  - JCheckBox()
  - JCheckBox(String)
  - JCheckBox(String, boolean)
  - boolean isSelected()
  - void setSelected(boolean)
  - String getLabel()
  - void setLabel(String)
  - boolean getState()
  - void setState(boolean)





# Elementos: JRadioButton

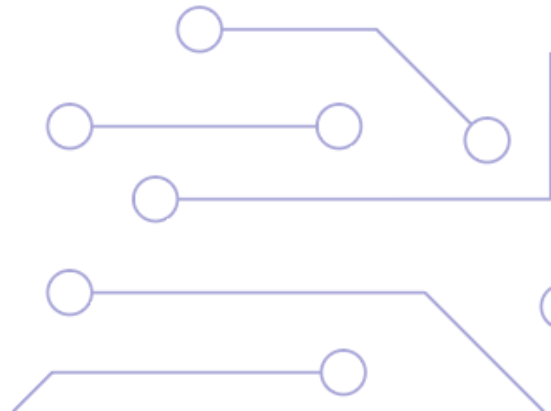
- Se utiliza para indicar botones que seleccionan una sola opción de un conjunto de botones.
- Métodos:
  - JRadioButton()
  - JRadioButton(String)
  - JRadioButton(String, boolean)
  - JRadioButton(Icon)
  - JRadioButton(Icon, boolean)
  - JRadioButton(String, Icon)
  - JRadioButton(String, Icon, boolean)
  - boolean isSelected()
  - void setSelected(boolean)



# Elementos: JRadioButton

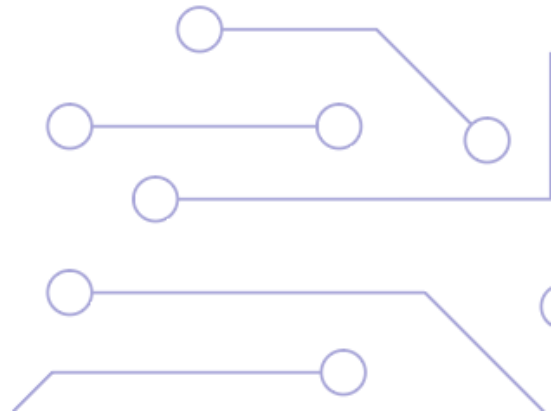
- El grupo de botones se llama ButtonGroup.
  - Tiene un borde y un nombre.
  - El método add se utiliza para agregar JRadioButtons al grupo.

```
ButtonGroup group = new ButtonGroup();  
group.add(radiobutton1);  
group.add(radiobutton2);  
group.add(radiobutton3);
```



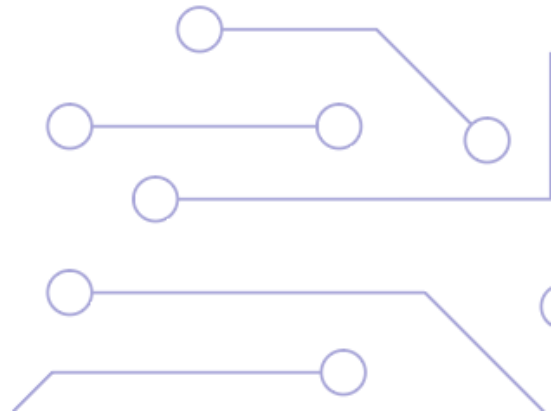
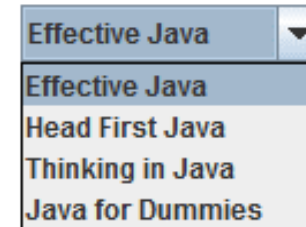
# Elementos: JComboBox

- Crea un menú que se abre de arriba a abajo y muestra diferentes opciones.
  - Solo se puede seleccionar una opción y puede ser editable o no.
- Métodos:
  - JComboBox()
  - JComboBox(Object[])
  - JComboBox(ComboBoxModel)
  - Object getSelectedItem()
  - int getSelectedIndex( )
  - void addItem(Object)



# Elementos: JComboBox

- Metodos:
  - void remove(int)
  - void removeAll()
  - void setSelectedIndex(int)
  - Object getItemAt(int)
  - int getItemCount()
  - void setEditable(boolean)
  - void setModel(ComboBoxModel)



# Elementos: JComboBox

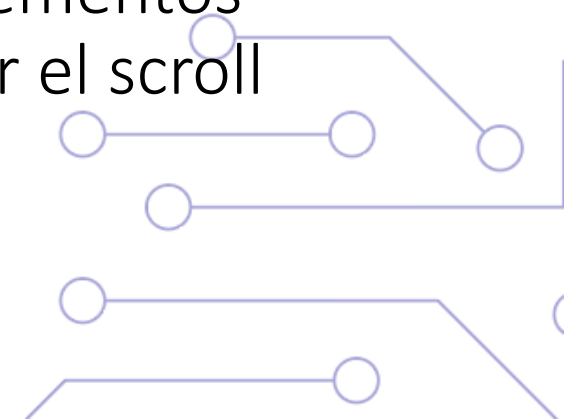
```
MiVentana()  
{  
    super("Combo box");  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    setPreferredSize(new Dimension(300,200));  
    setLayout(new FlowLayout(FlowLayout.CENTER));  
    String[] months={"enero", "febrero", "marzo", "abril", "mayo", «junio", «julio", "agosto", "septiembre",  
"octubre", "noviembre", "diciembre"};  
    JComboBox combo = new JComboBox(months);  
    combo.setSelectedIndex(4);  
    add(combo);  
    pack();  
    setVisible(true);  
}
```

# Elementos: JComboBox

```
ComboClase(){  
    JComboBox combo = new JComboBox();  
    DefaultComboBoxModel model = new DefaultComboBoxModel();  
    combo.setModel(model);  
    model.addElement(new Talde("Elegir", "nold"));  
    model.addElement(new Talde("Metallica", "met"));  
    model.addElement(new Talde("Megadeth", "meg"));  
    model.addElement(new Talde("Sepultura", "sep"));  
    model.addElement(new Talde("Pantera", "pan"));  
    combo.setBounds(10, 10, 120, 30);  
    this.getContentPane().add(combo);  
    pack();  
    setVisible(true);  
}
```

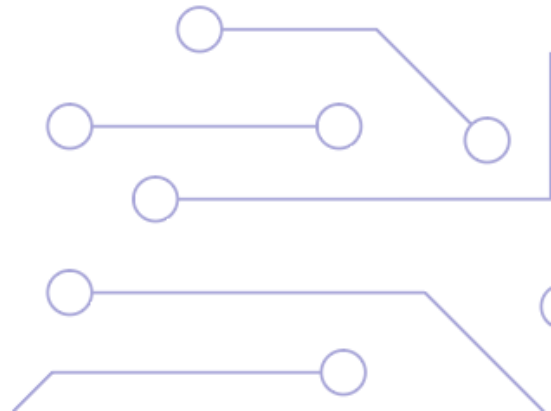
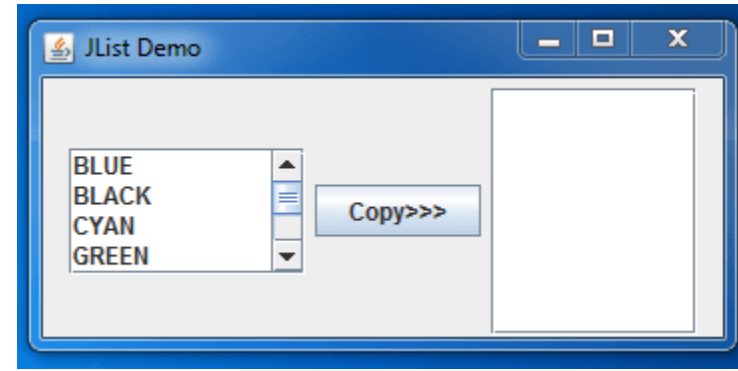
# Elementos: JList

- Es una lista que muestra varios elementos y el usuario puede seleccionar uno o más.
- Hay tres modos de selección:
  - Selección única
  - Selección de un solo rango
  - Selección de varios rangos
- Si su tamaño es demasiado pequeño para la cantidad de elementos que contiene, debe agregarse a un JScrollPane para permitir el scroll (desplazamiento) .



# Elementos: JList

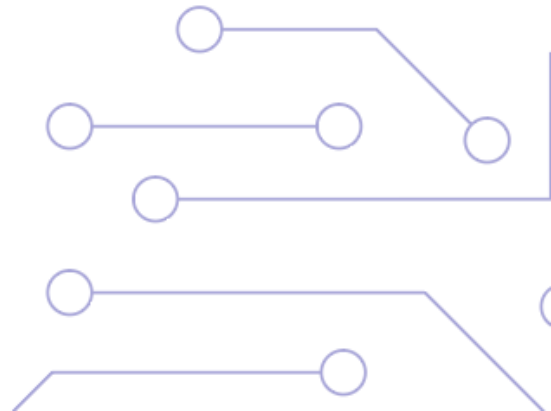
- Métodos:
  - JList()
  - JList(Object[])
  - JList(Vector)
  - JList(ListModel)
  - void setVisibleRowCount(int)
  - void setSelectedIndex(int)
  - void setSelectedIndices(int[])
  - boolean isIndexSelected(int)
  - void setListData (Vector)
  - void setModel(ListModel)





# Elementos: JList

- Métodos:
  - void setSelectionMode(int selectionMode)
    - Donde las opciones son ListSelectionModel.SINGLE\_SELECTION, ListSelectionModel.SINGLE\_INTERVAL\_SELECTION y ListSelectionModel.MULTIPLE\_INTERVAL\_SELECTION
  - void remove(int)
  - void removeAll()
  - int getSelectedIndex()
  - Object getSelectedValue()
  - int[] getSelectedIndices()
  - Object[] getSelectedValues()
  - Vector getListData()
  - ListModel getModel()



# Elementos: JList

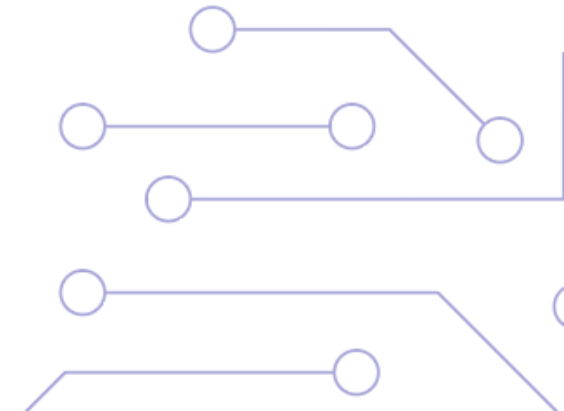
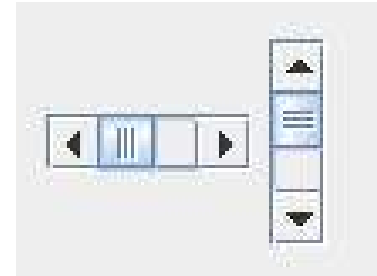
```
ListaClase()  
{  
    DefaultListModel<String> listaPaises = new DefaultListModel<>();  
    listaPaises.addElement("Francia");  
    listaPaises.addElement("Alemania");  
    listaPaises.addElement("Espanya");  
    listaPaises.addElement("Italia");  
    listaPaises.addElement("Grecia");  
    listaPaises.addElement("Portugal");  
    listaPaises.addElement("Belgica");  
    listaPaises.addElement("Irlanda");  
  
    add(new JList<>(listaPaises));  
  
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    this.setTitle("Ejemplo JList");  
    this.setSize(400, 150);  
    this.setVisible(true);  
}
```

# Elementos: JList

```
ListaClase()  
{  
    DefaultListModel<String> listaPaises = new DefaultListModel<>();  
    listaPaises.addElement("Francia");  
    listaPaises.addElement("Alemania");  
    listaPaises.addElement("Espanya");  
    listaPaises.addElement("Italia");  
    listaPaises.addElement("Grecia");  
    listaPaises.addElement("Portugal");  
    listaPaises.addElement("Belgica");  
    listaPaises.addElement("Irlanda");  
    JList<String> lista = new JList<>(listaPaises);  
    add(new JScrollPane(lista));  
    lista.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);  
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    this.setTitle("Ejemplo JList");  
    this.setSize(400, 150);  
    this.setVisible(true);  
}
```

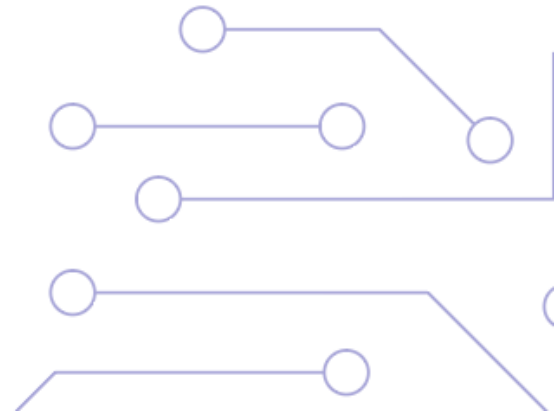
# Elementos: JScrollBar

- Un elemento que se mueve horizontal o verticalmente y adopta diferentes valores.
- Métodos:
  - JScrollBar()
  - JScrollBar(int orientacion)
  - JScrollBar(int orientacion, int valor, int anchura, int min, int max)
  - setMaximum(int max)
  - setMinimum(int min)
  - setUnitIncrement(int Unidaddeincremento)
  - setValue(int value)
    - Todos estos métodos también tienen una versión getXXX.



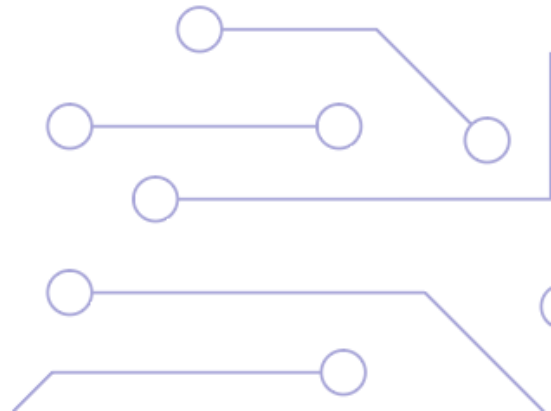
# Elementos: JLabel

- Es un elemento que representa texto no editable, una imagen o ambos.
  - Se denomina etiqueta.
- Métodos:
  - JLabel()
  - JLabel(String)
  - JLabel(String, LEFT|RIGHT|CENTER)
  - JLabel(Icon)
  - JLabel(String,Icon)
  - String getText()
  - void setText(String text)
  - void setIcon(Icon)
  - Icon getIcon()



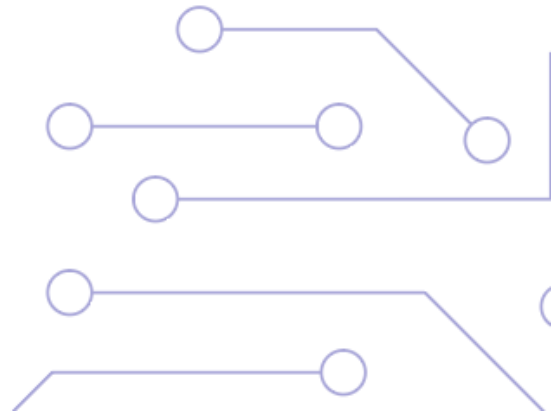
# Elementos: JTextComponent

- Incluye elementos para crear diferentes cuadros de texto.
- Todos estos elementos comparten algunos métodos.
  - String getText()
  - void setText(String)
  - boolean isEditable()
  - void setEditable(boolean)
  - getSelectedText()
  - select(int,int)
  - selectAll()
  - getSelectionStart()
  - getSelectionEnd()
  - setSelectionStart(int)
  - setSelectionEnd(int)



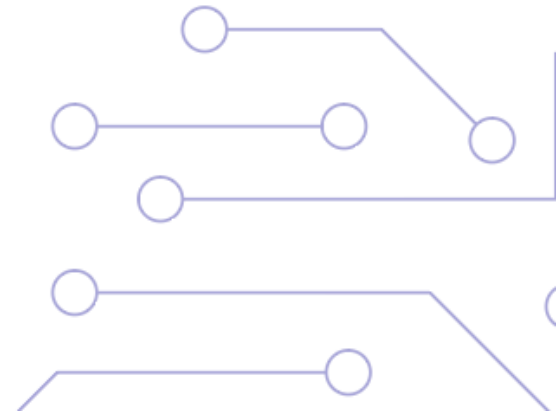
# Elementos: JTextField

- Añade un cuadro de texto editable.
- Métodos:
  - JTextField()
  - JTextField(String)
  - JTextField(String, int)
  - String getText()
  - void setText(String text)
  - int getColumns()
  - void setColumns (int)



# Elementos: JPasswordField

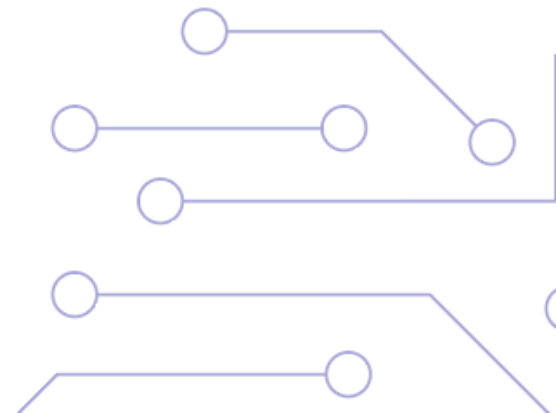
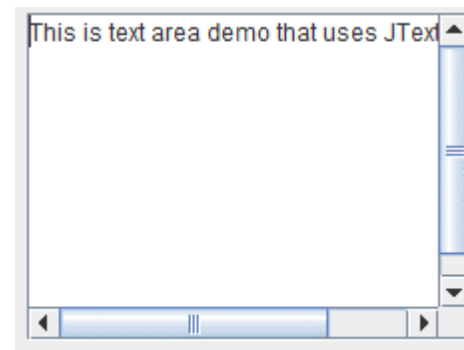
- El mismo elemento que el anterior, pero no se puede ver lo escrito.
- Métodos:
  - `char[] getPassword()`: este método se utiliza en lugar de `getText()`.
  - `setEchoChar('char')`





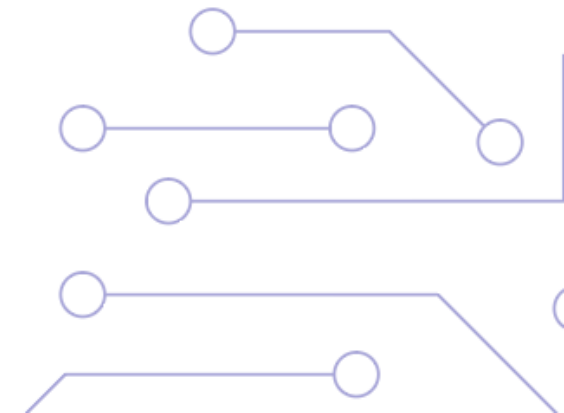
# Elementos: JTextArea

- Se utiliza cuando se necesita mostrar o escribir un texto de varias líneas.
- Si es demasiado pequeño para el tamaño del texto, debe agregarse a un JScrollPane para que tenga scroll (desplazamiento); de lo contrario, no se verá bien.
- Métodos:
  - JTextArea()
  - JTextArea(String)
  - JTextArea(int, int)
  - JTextArea(String,int,int)
  - String getText()
  - void setText(String text)



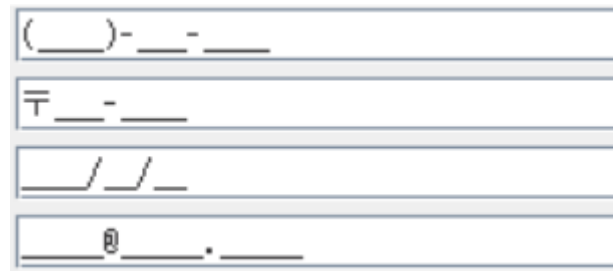
# Elementos: JTextArea

- Métodos:
  - int getColumns()
  - void setColumns(int)
  - int getRows()
  - void setRows(int)
  - void append(String)
  - void insert(String, int)
  - void replaceRange(String, int, int)
  - void setLineWrap(boolean)



# Elementos: JFormattedTextField

- Es similar al elemento JTextField, pero se puede definir una máscara o formato específico para limitar los caracteres que se pueden introducir.
  - Ideal para fechas, porcentajes, direcciones de correo electrónico, etc..



Four examples of JFormattedTextField masks are shown in a vertical stack:

- Phone number: ( ) - - -
- Time: - : -
- Date: / / -
- Email: @ . -

